

Laboratorio 12 · Percona Backup for MongoDB

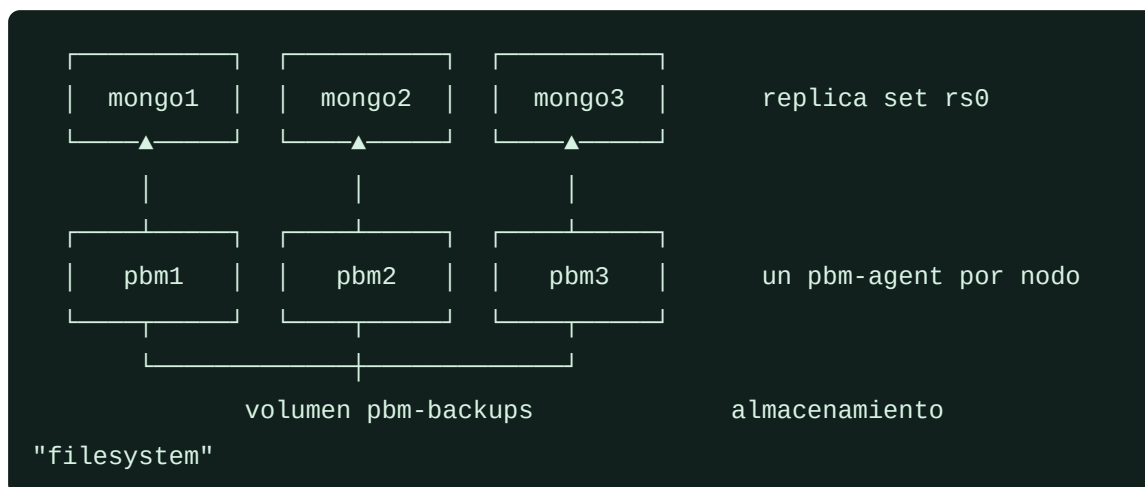
backup lógico y recuperación a un instante (PITR)

Duración: 35 minutos · Carpeta de trabajo: entorno/

Objetivos

Vas a desplegar Percona Backup for MongoDB (PBM) sobre el replica set, a lanzar y listar backups, a activar la recuperación a un instante concreto (*Point-In-Time Recovery*, PITR) y a usarla para deshacer un borrado accidental volviendo al segundo exacto anterior al error.

Arquitectura



Los agentes se coordinan a través de colecciones de control que PBM crea en la base de datos `admin` del propio replica set. El CLI `pbm` no habla con los agentes directamente: escribe órdenes en esas colecciones y los agentes las ejecutan. Con MongoDB Community, PBM hace backups **lógicos**; los backups físicos e incrementales requieren Percona Server for MongoDB.

Paso 1 · Usuario de PBM

```
cd entorno
docker compose up -d
docker compose exec toolbox bash
mongosh "$RS"
```

```

> use admin
> db.createRole({
  role: "pbmAnyAction",
  privileges: [ { resource: { anyResource: true }, actions: [
"anyAction" ] } ],
  roles: []
})
> db.createUser({
  user: "pbmuser", pwd: "PbmCurso2026",
  roles: [
    { db: "admin", role: "readWrite", collection: "" },
    { db: "admin", role: "backup" },
    { db: "admin", role: "clusterMonitor" },
    { db: "admin", role: "restore" },
    { db: "admin", role: "pbmAnyAction" }
  ]
})
> exit

```

Sal del toolbox (`exit`).

Paso 2 · Arrancar los agentes y configurar el almacenamiento

```

docker compose -f compose.yml -f compose.pbm.yml up -d
docker compose -f compose.yml -f compose.pbm.yml ps

```

Los contenedores `pbm1`, `pbm2` y `pbm3` tienen nombre fijo, así que a partir de aquí basta con `docker exec`:

```

docker exec pbm1 pbm config --file /scripts/pbm-config.yaml
docker exec pbm1 pbm status

```

En `pbm status` deben aparecer los tres agentes con estado `[OK]`. Si alguno no aparece, revisa su log con `docker logs pbm2`.

Paso 3 · Primer backup

```

docker exec pbm1 pbm backup --wait
docker exec pbm1 pbm list
docker exec pbm1 ls -lh /backups

```

Mientras se ejecuta, ¿qué nodo está haciendo el trabajo? Compruébalo con `docker exec pbm1 pbm logs --tail 20`.

▼ Ver solución

PBM elige preferentemente un **secundario** sano para leer los datos, para no cargar al primario. En el log verás qué agente (`rs0/mongoX:27017`) ha tomado el trabajo. El backup queda en `/backups` con un fichero de metadatos `.pbm.json` y un directorio con los datos comprimidos (en el formato elegido, `s2` en nuestra configuración).

Paso 4 · Activar PITR

```
docker exec pbm1 pbm config --set pitr.enabled=true
docker exec pbm1 pbm status
```

A partir de ahora PBM copia continuamente el oplog en trozos (*chunks*). En nuestra configuración (`oplogSpanMin: 1`) cada minuto; el valor por defecto es 10 minutos. Espera un par de minutos y comprueba que aparece un rango PITR:

```
docker exec pbm1 pbm list
```

Paso 5 · El accidente

Entra en el toolbox y simula la jornada: algunas escrituras normales, anotar la hora exacta y, después, un borrado catastrófico.

```
docker compose exec toolbox bash
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.insertOne({ nota: "pedido importante de las", ts: new Date() })'
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.countDocuments({ ciudad: "Madrid" })'
sleep 5
date -u +%Y-%m-%dT%H:%M:%S | tee /dumps/antes-del-accidente.txt
sleep 5
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.deleteMany({ ciudad: "Madrid" })'
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.countDocuments({ ciudad: "Madrid" })'
exit
```

Anota la hora que se ha mostrado (está en UTC). Espera dos minutos para que PBM haya copiado el oplog que cubre ese momento y comprueba que el rango PITR la incluye:

```
docker exec pbm1 pbm list
```

Paso 6 · Recuperación a un instante

Antes de restaurar hay que **desactivar PITR** (PBM no permite restaurar mientras está copiando el oplog):

```
docker exec pbm1 pbm config --set pitr.enabled=false
docker exec pbm1 pbm restore --time="2026-09-24T10:15:30" --wait      #
pon TU hora
```

Cuando termine, comprueba el resultado:

```
docker compose exec toolbox bash
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.countDocuments({ ciudad: "Madrid" })'
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").pedidos.findOne({ nota: /importante/ })'
exit
```

Tras una restauración, reactiva PITR y lanza un backup nuevo: la cadena de oplog anterior ya no es continua con el estado actual de los datos.

```
docker exec pbm1 pbm config --set pitr.enabled=true
docker exec pbm1 pbm backup --wait
docker exec pbm1 pbm list
```

1. ¿Ha vuelto el pedido "importante" que se insertó antes del accidente?
2. ¿Qué ha hecho PBM exactamente para llegar a ese instante?
3. ¿Qué ha pasado con las escrituras que la aplicación hizo **después** del accidente?

▼ Ver solución

PBM ha restaurado el último backup completo anterior a la hora indicada y, sobre él, ha reaplicado el oplog guardado por PITR **hasta ese segundo exacto**. El pedido importante vuelve porque se insertó antes; los pedidos de Madrid vuelven porque el borrado fue después. Pero todo lo que se escribió después de esa hora también ha desaparecido: una restauración PITR completa lleva **todo** el replica set a ese instante. En un incidente real se

valora si es mejor restaurar en un entorno aparte y recuperar solo los documentos borrados (PBM permite restauraciones selectivas con `--ns=tienda.pedidos`) o hacer una restauración completa y asumir la pérdida de lo posterior.

Para terminar

Deja PBM en marcha si quieres seguir experimentando, o páralo para liberar memoria antes del laboratorio de PMM:

```
docker compose -f compose.yml -f compose.pbm.yml stop pbm1 pbm2 pbm3
```